

Matthew Ives, Vihaan Madhavan, Charlie Price, Eric Xiang

22 July 2026

Implementing a Minimax Solver in Zero-Sum Games

Introduction

The games implemented in this project were the Halving Game, Tic-Tac-Toe, Connect Four, and Nim. Each game is a two-player, turn-based game in which players alternate choosing from a set of legal actions. In the Halving Game, the game begins with a randomly selected number of pieces between 15 and 50, inclusive. On each turn, a player may either subtract one from the total or halve the total; the player that removes the final piece wins. In Nim, players take turns removing objects from three separate piles. A player may remove any positive number of objects from one pile during a turn. Similar to the Halving Game, the player who removes the final object wins. In Connect Four, players take turns dropping pieces into one of four columns. Each player places one piece per turn, and the objective is to form a line of four matching pieces horizontally, vertically, or diagonally. In Tic-Tac-Toe, two players use the symbols X and O on a three-by-three grid. On each turn, a player places their symbol in an empty square. The first player to place three symbols in a horizontal, vertical, or diagonal line wins. In both Connect 4 and Tic-Tac-Toe, if the board becomes full without either player creating a line, the game ends in a tie. Tic-Tac-Toe is a deterministic, two-player, zero-sum game with no hidden information, meaning that both players can see the complete state of the board; and one player's advantage is the other player's disadvantage. All of these properties made Tic-Tac-Toe the ideal choice for more detailed analysis.

Strategy & Implementation

The game engine uses a basic Game class including the methods: “next”, “is_end”, “actions”, “successors”, “utility”, “start_state”, and “player”, which provides a common structure for every game engine we make. Every state contains both the current game position and a Boolean representing the active player. In Tic-Tac-Toe, a state is represented by the tuple: (board, player). The board is a 3x3 matrix containing "X", "O", or None. An action is a (row, column) tuple identifying an empty square that the player wants to put their symbol in. A terminal state occurs when the board is full or a player gets three in a row horizontally, vertically, or diagonally. An “X” victory has a utility of 1000, an “O” victory has a utility of -1000, and a tie has a utility of 0.

The recursive minimax method evaluates every possible future combination of the current game state. It returns a tuple containing the best possible utility and the action that produces that result. When the method reaches an end state, it returns the utility of that state and None because no additional move is available.

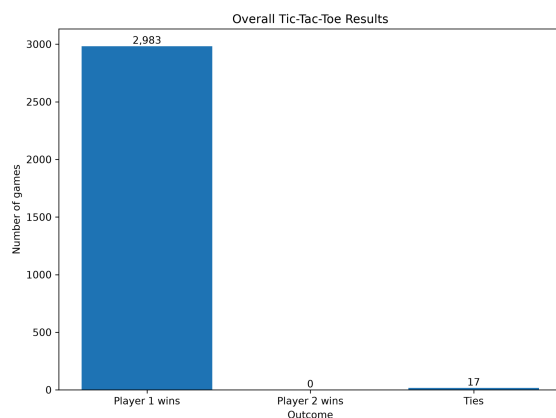
For the maximizing player, minimax begins with negative infinity and checks every possible action. It recursively evaluates the resulting state and returns the action with the highest utility. The minimizing player begins with positive infinity and keeps the action with the lowest score. After each recursive call, the maximizing player Boolean is reversed so the algorithm alternates between maximizing “X”’s result and minimizing “O”’s result.

The main enhancement is memoization. Since the board uses mutable lists, state_to_key() converts the board into nested tuples so it can be used as a dictionary key. Each evaluated state is cached in a dictionary. When minimax reaches a previously evaluated state, it immediately returns the stored result instead of recalculating the remaining game tree. The program also calls

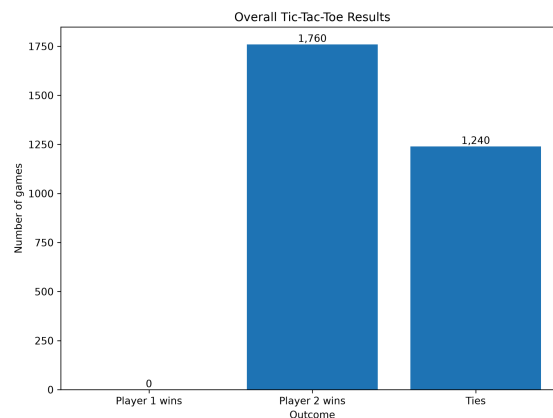
minimax on the starting state before beginning the simulations to start with cached moves and run more efficiently.

Simulation & Design

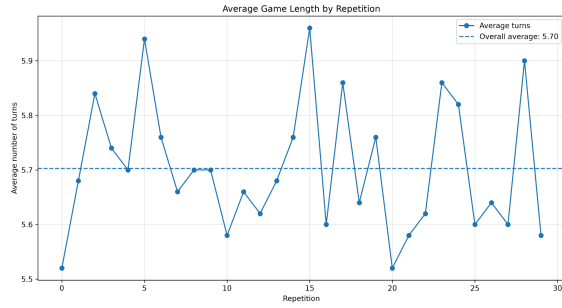
For our testing, we ran a total of 6,000 simulations. Our testing was split into chunks of 100 simulations. The agents used in the simulations were a maximizing agent and a completely random agent. In the first 3,000 simulations, the maximizing agent was Player 1 and had the first move. The agents used in the second 3,000 simulations swapped their order, and Player 1 always had the first move. The simulations recorded whether the minimax agent won, lost, or tied and how many turns the game lasted. For every 100 simulations, we stored the data in a CSV file with the headers: repetition, p1_wins, p2_wins, ties, and avg_turns. To display the data cleanly in a graph, we wrote a pandas function to create the data visualization. We did this twice for both first move variants.



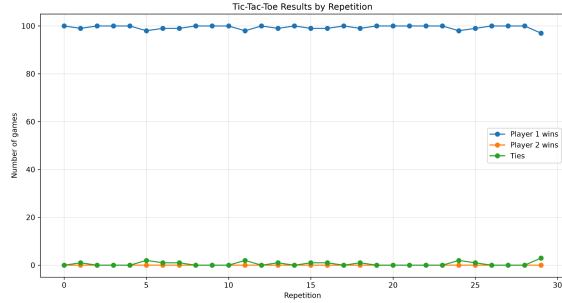
(Figure 1: simulations with minimax agent as Player 1)



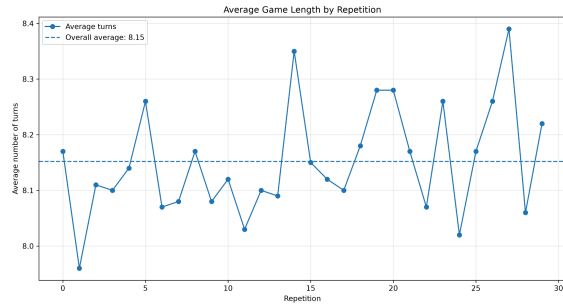
(Figure 2: simulations with random agent as Player 1)



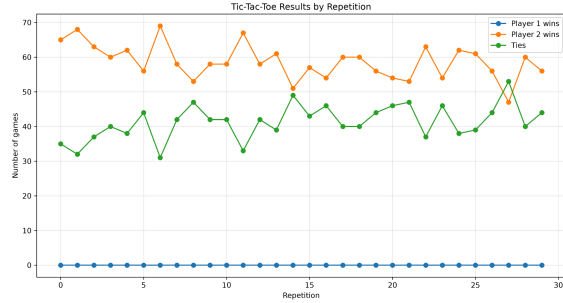
(Figure 3: Average game length with minimax agent starting)



(Figure 5: Results by repetition with minimax agent starting)



(Figure 4: Average game length with random agent starting)



(Figure 6: Results by repetition with random agent starting)

Results Discussion

Across the 3,000 Tic-Tac-Toe games in which the minimax agent moved first, it won 99.43% of the games and never lost (Figures 1 & 5). When the random agent moved first, the minimax agent won 58.67% of the games and never lost (Figures 2 & 6). These results demonstrate that minimax consistently outperformed the random policy and that perfect play in Tic-Tac-Toe guarantees at least a tie. However, the large difference in minimax's win rate shows that perfect play guarantees at least a tie. When minimax moved first, it controlled the opening and had more opportunities to take advantage of mistakes made by the random agent. When the random agent moved first, some sequences of random moves prevented minimax from creating a forced win. In those games, minimax selected moves that guaranteed a tie rather than risking a loss. When the minimax agent started first, it was easily able to take advantage of the random mistakes the random agent made; it controlled the game from the very start (Figure 3).

Conversely, when the random agent made the first move, the minimax agent had to react to the random move instead of starting out aggressively and trying to win (Figure 4). The greater number of moves and ties when the random agent started are directly related; the minimax agent spent more time on defense and, in turn, spent more moves trying not to lose rather than trying to win. Ultimately, the minimax agent was very successful since it always won or tied; and it demonstrated an application of the minimax algorithm.